AI ASSISTANT CODING
ASSIGNMENT-19.2

Name: G.vishnu

Enrollment no: 2303A52194

Batch: 34

Semester: VI

Branch: CSE(AIML)

Lab 19 – Code Translation: Converting Between Programming Languages

Task 1 – Python to Java Conversion Task:

Convert a Python program that performs file handling operations into an equivalent Java program using AI assistance.

Instructions:

- Prompt AI to translate a Python script that reads from and writes to a file into Java.
- Ensure proper handling of file streams, exceptions, and input/output formatting.
- Verify that the Java program produces the same file content and console output as the Python version.
- Test both implementations with sample input files.

Expected Output:

A correctly translated and executable Java program that performs the same file handling operations as the original Python script.

main.py

```python
try:
    print("Enter file content (type END to stop):")
    lines = []
    while True:
        line = input()
        if line == "END":
            break
        lines.append(line)

    data = "\n".join(lines)
    print("\nInput File Content:")
    print(data)

    # Process data
    processed_data = data.upper()

    # Simulate writing to file
    print("\nWriting to output file...")

    print("\nOutput File Content:")
    print(processed_data)
except Exception as e:
    print("Error:", e)
```

Output

```
Enter file content (type END to stop):
Hello World
Welcome to File Handling
END

Input File Content:
Hello World
Welcome to File Handling

Writing to output file...

Output File Content:
HELLO WORLD
WELCOME TO FILE HANDLING

=== Code Execution Successful ===
```

Main.java

```java
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        StringBuilder data = new StringBuilder();
        try {
            System.out.println("Enter file content (type END to stop):");
            while (true) {
                String line = scanner.nextLine();
                if (line.equals("END")) {
                    break;
                }
                data.append(line).append("\n");
            }
            System.out.println("\nInput File Content:");
            System.out.print(data.toString());
            String processedData = data.toString().toUpperCase();
            System.out.println("\nWriting to output file...");
            System.out.println("\nOutput File Content:");
            System.out.print(processedData);
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            scanner.close();
        }
    }
}
```

Output

```
Enter file content (type END to stop):
Hello World
Welcome to File Handling
END

Input File Content:
Hello World
Welcome to File Handling

Writing to output file...

Output File Content:
HELLO WORLD
WELCOME TO FILE HANDLING

=== Code Execution Successful ===
```

Task 2 – C++ to Python Function Conversion Task:

Translate a C++ program that implements a sorting algorithm into Python using AI assistance.

Instructions:

- Provide AI with a C++ implementation of a sorting technique (e.g., Bubble Sort or Selection Sort).
- Ask AI to generate an equivalent Python implementation.
- Ensure logical correctness and validate results using multiple test cases.
- Confirm that the Python version maintains the same sorting behavior.

Expected Output:

A correct and executable Python program equivalent to the given C++ sorting implementation.

main.cpp

```cpp
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n) {
    for(int i = 0; i < n-1; i++) {
        for(int j = 0; j < n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                // swap
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr)/sizeof(arr[0]);

    bubbleSort(arr, n);

    cout << "Sorted array: ";
    for(int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Output

```
Sorted array: 11 12 22 25 34 64 90

=== Code Execution Successful ===
```

main.py

```python
# Python Equivalent of Bubble Sort

def bubble_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                # swap
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

# Test the function
arr = [64, 34, 25, 12, 22, 11, 90]

bubble_sort(arr)

print("Sorted array:", end=" ")
for num in arr:
    print(num, end=" ")
```

Output

```
Sorted array: 11 12 22 25 34 64 90
=== Code Execution Successful ===
```

Task 3 – SQL to Pandas Query Task:

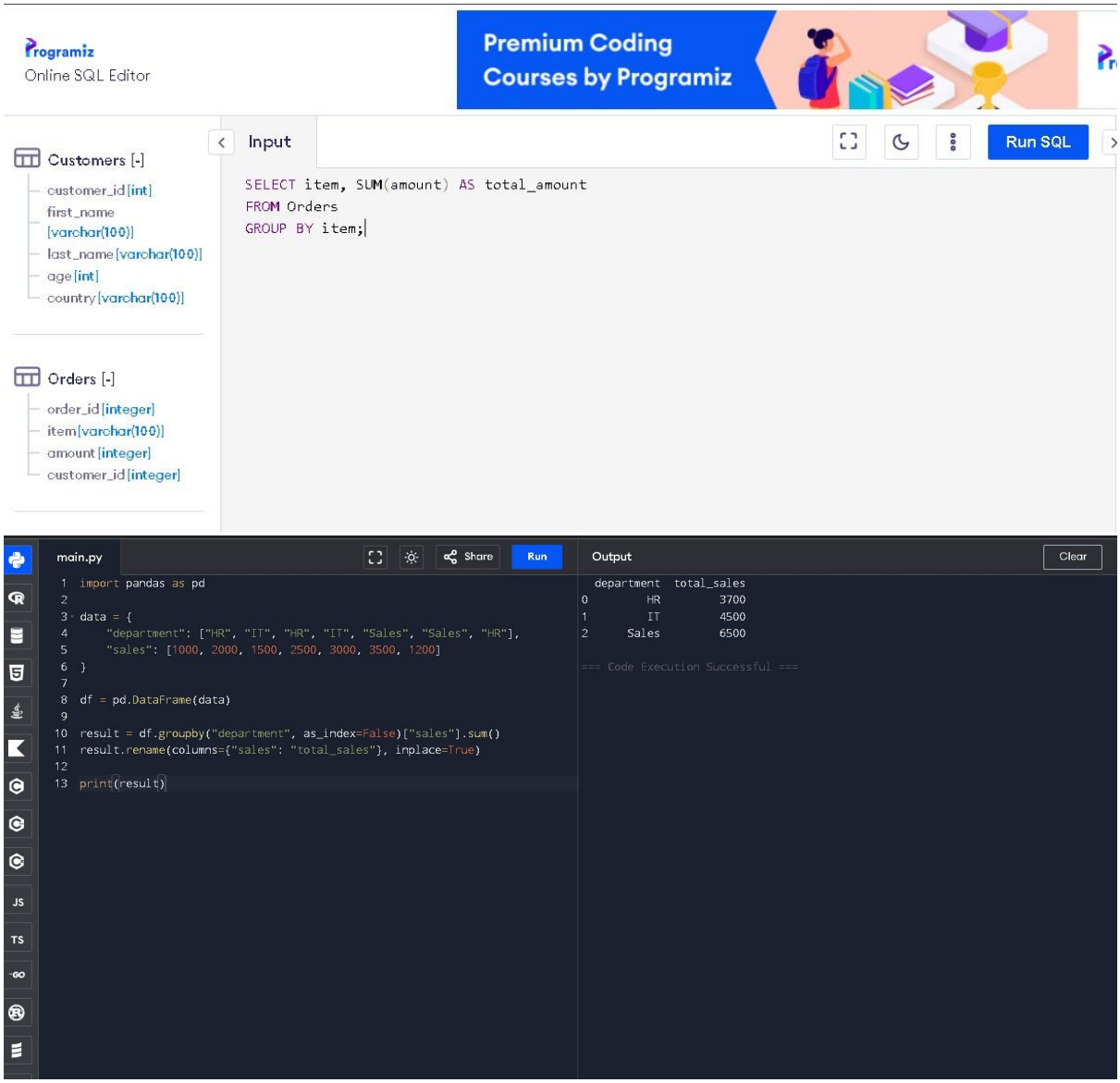
Convert a SQL query involving GROUP BY and aggregate functions into an equivalent Pandas Data Frame operation.

Instructions:

- Provide AI with a SQL query that calculates aggregate values (e.g., total sales per department).
- Prompt AI to generate equivalent Pandas code using groupby() and aggregation functions.
- Test the generated code using a sample dataset.
- Verify that both SQL and Pandas outputs match.

Expected Output:

An accurate Pandas Data Frame operation that produces results equivalent to the given SQL query with aggregation.



Task 4 – Real-Time Application: Multi-Language Snippet Converter Scenario:

Students will select a practical real-world application requiring the same logic in two different programming languages.

Example:

- Converting a REST API implementation from Python (Flask) to Java (Spring Boot).
- Translating a data processing script from Python to C++ for performance optimization.

Instructions:

- Provide AI with source code in one programming language.
- Prompt AI to convert the code into another chosen language.
- Execute and test both versions to ensure identical functionality.
- Document differences observed in syntax, libraries, and execution behavior.

Expected Output:

Functionally equivalent and executable code snippets implemented in two different programming languages, along with documented observations.

Welcome | app.py

C: > Users > hp > Downloads > app.py > home

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/')
def home():
    return "Welcome to API"

@app.route('/sum', methods=['GET'])
def calculate_sum():
    a = int(request.args.get('a'))
    b = int(request.args.get('b'))
    return jsonify({"result": a + b})
if __name__ == '__main__':
    app.run(debug=True)
```

PROBLEMS | DEBUG CONSOLE | TERMINAL | PORTS

```
Requirement already satisfied: itsdangerous>=2.2.0 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask) (3.0.2)
Requirement already satisfied: werkzeug>=3.1.0 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flask) (3.1.6)
Requirement already satisfied: colorama in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from click>=8.1.3->flask) (0.4.6)
PS C:\Users\hp>
fwd-i-search: _
```

powers... | Python

Ln 5, Col 16 | Spaces: 4 | UTF-8 | CRLF | Python | Python 3.13 (64-bit) | Go Live

127.0.0.1:5000

Welcome to API

main.cpp | Share | Run

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
pair<vector<int>, int> processNumbers(vector<int> nums) {
    sort(nums.begin(), nums.end());
    int total = 0;
    for (int n : nums) {
        total += n;
    }
    return {nums, total};
}
int main() {
    vector<int> nums;
    int x;

    cout << "Enter numbers: ";
    while (cin >> x) {
        nums.push_back(x);
        if (cin.peek() == '\n') break;
    }
    auto result = processNumbers(nums);
    cout << "Sorted: ";
    for (int n : result.first) {
        cout << n << " ";
    }
    cout << "\nSum: " << result.second;
    return 0;
}
```

Output | Clear

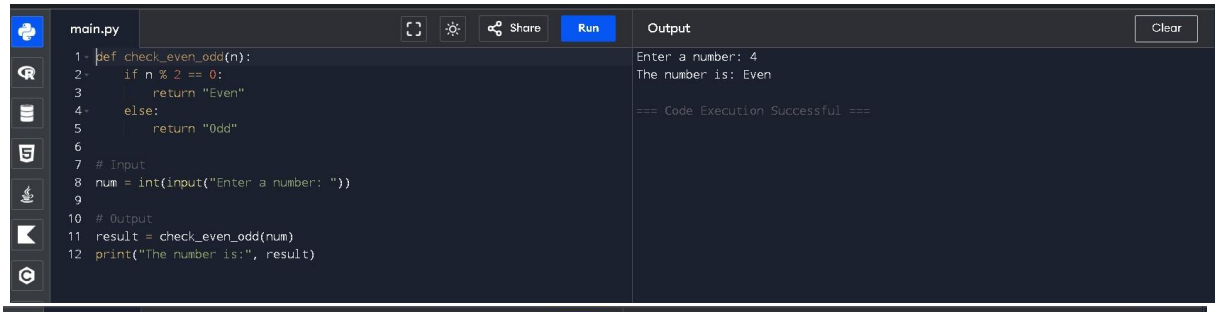
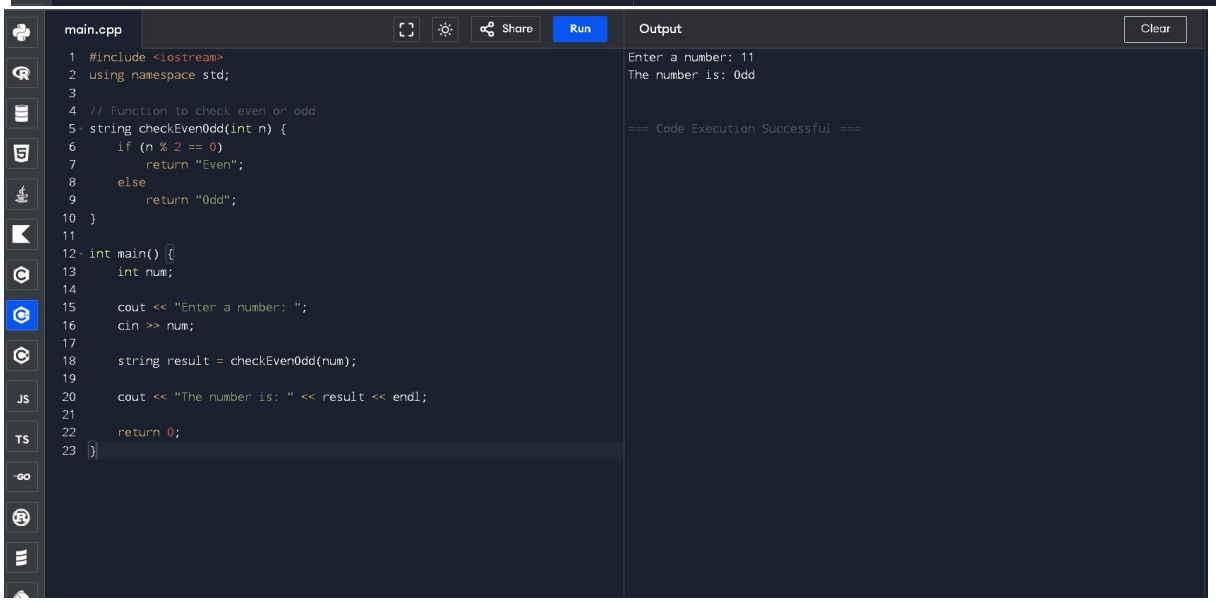
```
Enter numbers: 5 2 9 1
Sorted: 1 2 5 9
Sum: 17

=== Code Execution Successful ===
```

Task -5: (Basic Function Translation (Python to C++)

Translate a simple Python function that checks whether a number is even or odd into C++ using AI assistance.

Instructions:

- Provide AI with a basic Python function that determines if a number is even or odd.
- Prompt AI to generate equivalent C++ code.
- Ensure correct syntax, input handling, and output formatting.
- Compile and run the C++ program to verify it produces the same result as the Python version.
- Test with at least three different input values.

Expected Output:

A simple and executable C++ program equivalent to the given Python function, correctly identifying whether a number is even or odd for all test cases.